

Demonstração e Apresentação de Desempenho de Modelos de Difusão de Contaminantes em Água

Fernando Daniel Marcelino¹, Lucas Moura da Silva¹,
Marcos Vinicius Gasparoto Bauab¹

¹Universidade Federal de São Paulo – Campus São José dos Campos
Avenida Cesare Mansueto Giulio Lattes, 1201 -Eugênio de Mello
São José dos Campos/SP - CEP: 12247-014

{fernando.marcelino, lucas.silva16, marcos.bauab}@unifesp.br

Resumo. A contaminação dos recursos hídricos por poluentes representa um dos mais graves problemas ambientais da atualidade, afetando significativamente a saúde pública e a biodiversidade global, com impacto especialmente severo em países em desenvolvimento. A difusão de contaminantes em corpos d'água é um fenômeno complexo que pode ser modelado por equações diferenciais parciais. Neste trabalho, é apresentado um modelo aproximado de difusão de contaminantes em água, implementado na linguagem C e paralelizado utilizando OpenMP, CUDA e MPI. Foram realizados testes de desempenho para avaliar o impacto da paralelização no tempo de execução do modelo, e os resultados obtidos mostraram que a paralelização foi eficaz na redução do tempo de execução, para algumas metodologias. Assim, destaca-se a relevância do uso de arquiteturas paralelas baseadas em GPUs para a solução de problemas ambientais que demandam alto poder computacional.

1. Introdução

A contaminação da água é um problema ambiental crítico que afeta a saúde pública, a biodiversidade e a economia global. Trata-se de uma questão que preocupa grandes órgãos nacionais e internacionais, como a Organização das Nações Unidas (ONU). Em 2015, os países-membros da ONU comprometeram-se com um apelo universal à ação para erradicar a pobreza, proteger o planeta e garantir que, até 2030, todas as pessoas desfrutem de paz e prosperidade. Contudo, em virtude do aumento das atividades industriais, agrícolas e urbanas, a quantidade de poluentes lançados nos corpos d'água tem crescido significativamente, tornando essencial o desenvolvimento de modelos eficazes para prever a difusão de contaminantes.

Diante desse cenário, modelos computacionais de difusão de contaminantes em água são ferramentas fundamentais para entender e prever o comportamento de poluentes. Esses modelos permitem simular a dispersão de substâncias químicas, biológicas e físicas em rios, lagos, aquíferos e oceanos, fornecendo informações valiosas para a gestão ambiental e a tomada de decisões. No entanto, tais simulações frequentemente demandam alto custo computacional, especialmente quando realizadas em grande escala ou com alta precisão.

Para enfrentar esse desafio computacional, este trabalho explora três abordagens de computação paralela: a API OpenMP (*Open Multi-Processing*) para processamento em

CPU *multicore*, a arquitetura *CUDA* (*Compute Unified Device Architecture*) da NVIDIA para processamento em GPUs, e o padrão *MPI* (*Message Passing Interface*) para sistemas distribuídos. A escolha dessas tecnologias justifica-se pela sua complementaridade: enquanto o *OpenMP* simplifica a paralelização em sistemas com memória compartilhada, o *CUDA* explora o paralelismo massivo de GPUs para cálculos matriciais intensivos, e o *MPI* permite escalabilidade em clusters heterogêneos, onde processos independentes comunicam-se por troca de mensagens [Gropp et al. 2014, MPI Forum 2021].

O objetivo deste trabalho é apresentar uma análise detalhada e comparativa entre quatro implementações do modelo de difusão de contaminantes: uma versão sequencial (*baseline*), uma versão paralela utilizando *OpenMP*, outra com *CUDA* e uma quarta com *MPI*. O modelo adapta-se às condições dos fluidos e de variáveis físicas, como volume amostral e coeficiente de difusão. A implementação base foi realizada em C/C++, e foram conduzidos testes extensivos de desempenho em diferentes configurações de *hardware*, incluindo ambientes distribuídos para avaliação do *MPI*. Os resultados são analisados em termos de tempo de execução, *speedup*, eficiência de paralelização e escalabilidade, destacando cenários onde cada tecnologia se mostra mais adequada.

2. Fundamentação Teórica

A fundamentação teórica deste trabalho aborda os conceitos essenciais e as ferramentas utilizadas na implementação do modelo matemático de difusão de contaminantes, com ênfase nas técnicas de programação paralela. Até o presente momento, os modelos propostos foram desenvolvidos utilizando o OpenMP, CUDA e MPI. Ambas as aplicações, diferem em suas abordagens de paralelização, mas compartilham o objetivo de otimizar o tempo de execução do modelo. A seguir, são apresentados os conceitos fundamentais de cada tecnologia, bem como suas características e aplicações.

2.1. OpenMP

A programação concorrente e distribuída pode ser feita de várias formas, e a principal maneira de se implementar é utilizando de *threads*. Para isso, também existem tecnologias consolidadas na academia e na indústria de software para aplicações de alto desempenho, como as *Application Programming Interface* (API) para programação paralela. Uma dessas tecnologias é o *OpenMP* (*Open Multi-Processing*), que é uma API de programação paralela de memória compartilhada, que permite a paralelização de laços de repetição, blocos de código e funções e outros [Chapman et al. 2007].

O *OpenMP* pode ser utilizada para as linguagens de programação Fortran, C/C++, possuindo formatos específicos em suas diretivas e funcionalidades. Sua programação é organizada igual à programas sequenciais, tornando a sintaxe muito fácil de ser entendido por usuários conencionais de aplicações lineares [Gropp and Lusk 2017].

Para se utizar dos benefícios da programação pela API *OpenMP*, é necessário a utilização de diretivas, que são instruções que são interpretadas pelo compilador, e que são utilizadas para a paralelização de trechos de código. As diretivas em C/C++ são precedidas por:

```
// Em C/C++
#pragma omp
```

```
// Em Fortran
!$omp
```

Após a declaração do formato descrito, é preciso utilizar de parâmetros e diretivas para que se consigo utilizar a tecnologia. Algumas diretivas amplamente utilizadas são:

<code>parallel</code>	<code>shared</code>
<code>for</code>	<code>reduction</code>
<code>critical</code>	<code>schedule</code>
<code>atomic</code>	<code>barrier</code>
<code>private</code>	<code>num_threads</code>

A listagem completa das diretivas e métodos acessadas pela *API* pode ser encontrada nos guias de referências¹ da *OpenMP*.

2.1.1. Diretivas

As principais diretivas do OpenMP incluem:

- `parallel`: Cria uma região paralela onde múltiplas threads executam simultaneamente.
- `for`: Distribui as iterações de um loop entre threads disponíveis.
- `collapse(n)`: Combina múltiplos loops aninhados em uma única iteração paralela.
- `reduction`: Executa operações de redução (como soma, produto) de forma segura entre threads.
- `schedule`: Define como as iterações são distribuídas entre as threads:
 - `static`: Divide as iterações igualmente entre threads.
 - `dynamic`: Distribui iterações sob demanda.
 - `guided`: Usa blocos de tamanho decrescente.
- `private`: Cria cópias locais de variáveis para cada thread.
- `shared`: Permite que todas as threads acessem a mesma variável.
- `critical`: Define uma região que só pode ser executada por uma thread por vez.
- `barrier`: Sincroniza todas as threads em um ponto específico.

Estas diretivas são fundamentais para controlar a execução paralela e garantir o correto compartilhamento de dados entre threads.

2.2. CUDA

A computação convencional baseada em CPUs utiliza um modelo de processamento sequencial, no qual as instruções são executadas em uma ordem predefinida. Contudo, essa abordagem começou a ser complementada à medida que a demanda por processamento de dados e instruções aumentou, exigindo uma quebra de paradigma na programação. Ao longo dos anos, esse novo paradigma foi amplamente adotado em áreas como processamento de imagens, simulações físicas, aprendizado de máquina e outras aplicações que requerem elevado poder de processamento. Dessa forma, iniciaram-se os estudos e

¹O guia de referência da *Open MP API* é um documento que abrange todas as possíveis implementações realizadas pela tecnologia. Pode ser encontradas na *web* pelo endereço www.openmp.org/resources/refguides/

o desenvolvimento de tecnologias de processamento gráfico, conhecidas como *Graphics Processing Units* (GPUs).

Na década de 1970, começaram os primeiros desenvolvimentos voltados ao processamento gráfico, e, um pouco antes dos anos 2000, consolidaram-se estudos que deram origem à arquitetura das GPUs modernas. Com isso, arquiteturas heterogêneas GPU-CPU passaram a ser empregadas em aplicações de alto desempenho [Cheng et al. 2014].

Com o avanço das GPUs, identificou-se que elas poderiam ser utilizadas para programação de propósito geral, e não apenas para processamento gráfico. A NVIDIA foi uma das pioneiras no desenvolvimento de tecnologias para essa finalidade, introduzindo a *Compute Unified Device Architecture* (CUDA). Desenvolvida em 2006, a CUDA é uma plataforma de computação paralela que permite a programação de GPUs NVIDIA para aplicações de alto desempenho [NVIDIA Corporation 2012]. A programação em CUDA é realizada em C/C++, e a plataforma oferece um conjunto de ferramentas, manuais e bibliotecas que facilitam o desenvolvimento de aplicações paralelas.

2.2.1. Núcleos de Processamento

O processamento em CUDA é realizado por meio da execução de um *kernel*, que é uma função especial criada na CPU e executada na GPU. O *kernel* é executado por um grande número de *threads* em paralelo, sendo que cada *thread* é processada por um núcleo da GPU. Os núcleos de processamento são organizados em blocos, enquanto os blocos são dispostos em *grids*. A invocação de um *kernel* em CUDA utiliza uma notação especial, com os parâmetros envolvidos entre os delimitadores <<< e >>>, onde o primeiro parâmetro especifica o número de blocos e o segundo, o número de *threads* por bloco.

```
1  __global__ void VecAdd(float* A, float* B, float* C)
2  {
3      int i = threadIdx.x;
4      C[i] = A[i] + B[i];
5  }
6
7  int main()
8  {
9      ...
10     // Kernel invocation with N threads
11     VecAdd<<<1, N>>>>(A, B, C);
12     ...
13 }
```

Listagem 1. Exemplo de definição e invocação de um kernel CUDA para adição vetorial

O código apresentado na Listagem 1 ilustra um exemplo de definição e invocação de um *kernel* CUDA para adição vetorial. Nesse exemplo, utiliza-se apenas um bloco contendo N *threads*. O *kernel* recebe três vetores de ponto flutuante como parâmetros (A , B e C) e realiza a soma elemento a elemento dos vetores A e B , armazenando o resultado no vetor C . O índice para percorrer os elementos dos vetores é unidimensional; entretanto, em problemas clássicos como a multiplicação de matrizes, os índices podem ser bidimensionais ou até tridimensionais.

A organização hierárquica dos blocos e *threads* exige o cálculo dos índices para acessar corretamente os elementos das estruturas de dados. O acesso às *threads* é feito

utilizando as funções *threadIdx.x*, *threadIdx.y* e *threadIdx.z*, em conjunto com *blockIdx.x*, *blockIdx.y* e *blockIdx.z*, bem como *blockDim.x*, *blockDim.y* e *blockDim.z*. Essa combinação permite o mapeamento completo das *threads* processadas pela GPU.

Um exemplo esquemático da organização de blocos e *threads* em CUDA é apresentado na Figura 1. A figura ilustra a identificação de *threads* e blocos em uma grade de 2x2 blocos, onde cada bloco contém 4x4 *threads*.

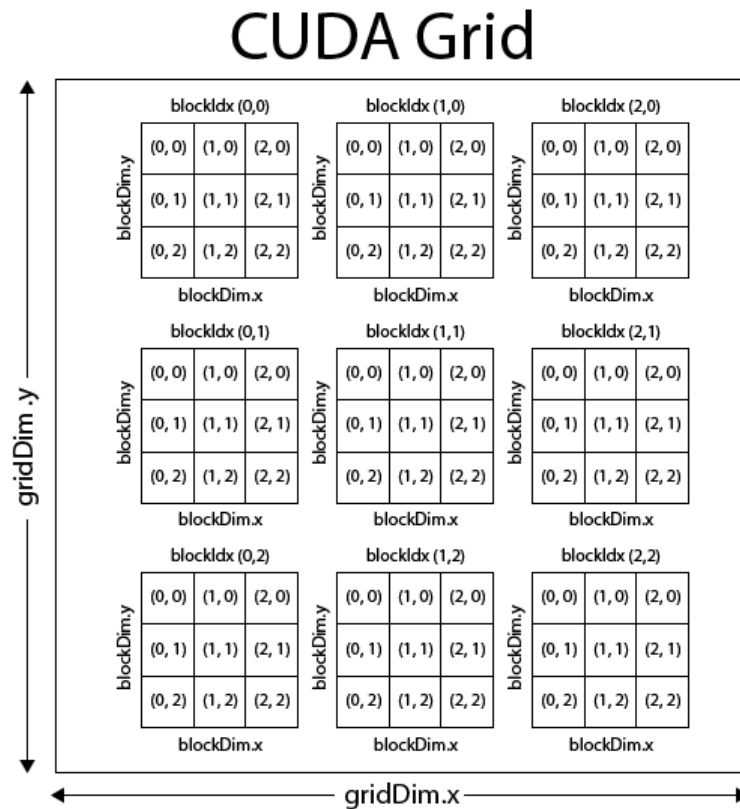


Figura 1. Esquema de identificação de *threads* e blocos em CUDA. [McKennon 2013]

2.2.2. GPU

A Unidade de Processamento Gráfico (*Graphics Processing Unit*, GPU) é um processador especializado no aceleração de processamento gráfico, especialmente em aplicações como renderização 3D e processamento de imagens. Com o avanço das arquiteturas computacionais, as GPUs evoluíram para plataformas de computação paralela de propósito geral, amplamente utilizadas em ciência de dados, aprendizado de máquina, simulações físicas e outras aplicações de alta performance, conhecidas como *General-Purpose Graphics Processing Unit* (GPGPU). [Owens et al. 2008]

Enquanto as CPUs são otimizadas para latência, as GPUs são projetadas para lidar com várias *threads* simultaneamente, devido à sua arquitetura massivamente paralela. Isso ocorre porque a GPU é composta por centenas ou milhares de núcleos organizados em múltiplos blocos, chamados de *Streaming Multiprocessors* (SMs). Cada SM é projetado para executar centenas de *threads* em paralelo, adotando um modelo de execução

baseado em *Single Instruction, Multiple Thread* (SIMT), que é uma das classificações da taxonomia de Flynn.

2.2.3. GPU T4

As GPUs T4, baseadas na arquitetura Turing da NVIDIA, representam um avanço significativo no uso de GPUs para computação de propósito geral e aprendizado de máquina. A T4 foi projetada como uma solução otimizada para data centers modernos, suportando cargas de trabalho mistas que incluem treinamento, inferência, virtualização e serviços baseados em inteligência artificial (IA).

A GPU T4 possui 49 KB de memória compartilhada, 4 MB de memória cache L2 e um tamanho de *warp* de 32 *threads*, ou seja, a quantidade de *threads* que podem ser usadas simultaneamente. O número máximo de *threads* por bloco e por multiprocessador é de 1024, e a GPU conta com 40 multiprocessadores. A dimensão máxima de *threads* por bloco é (1024, 1024, 64) nos eixos (x, y, z), enquanto o *grid* possui dimensão máxima de (2147483647, 65535, 65535).

A Figura 2 apresenta o esquemático da configuração da arquitetura da NVIDIA Tesla T4.



Figura 2. Arquitetura da GPU Tesla T4. [NVIDIA Corporation 2018]

2.3. Message Passing Interface (MPI)

Diferentemente do paradigma de programação realizado em CUDA, o MPI é uma biblioteca de comunicação de mensagens que permite a execução de programas paralelos

em sistemas distribuídos. A biblioteca possui semânticas para interfaces de programação em C e Fortran, com uma ampla gama de funções no padrão MPI. Com isso, processos podem estar sendo executados em sistemas operacionais diferentes, em diferentes arquiteturas de hardware e até mesmo em diferentes locais geográficos, diferentemente das outras abordagens encontradas para se fazer concorrência. O MPI é amplamente utilizado em supercomputadores e clusters de computadores, permitindo a comunicação entre processos independentes que executam em diferentes nós de um sistema distribuído. [Gropp et al. 2014]

Analogamente ao CUDA, o MPI também possui uma estrutura hierárquica, onde os processos são organizados em grupos chamados *communicators*. Cada *communicator* possui um identificador único, que é utilizado para a comunicação entre os processos. A comunicação entre processos é realizada por meio de funções de envio e recebimento de mensagens, como `MPI_Send` e `MPI_Recv`, que permitem a troca de dados entre processos de forma síncrona ou assíncrona. Portanto, antes mesmo de fazer a transmissão em paralelo dos processos, deve-se utilizar ferramentas de sincronização para garantir que a comunicação entre os processos seja feita de forma correta. A Listagem 2 apresenta um exemplo de código que utiliza a biblioteca MPI para realizar a inicialização, sincronizações e finalização dos processos.

```
1 #include <mpi.h>
2 #include <stdio.h>
3
4 int main(int argc, char** argv) {
5     int rank, size;
6
7     // Iniciando o ambiente MPI e obtendo o rank e o tamanho do comunicador conforme as entradas
8     MPI_Init(&argc, &argv);
9
10    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11    MPI_Comm_size(MPI_COMM_WORLD, &size);
12
13    // Procedimento a ser realizado por cada processo
14    printf("Hello, world! I am %d of %d\n", rank, size);
15
16    // Finalizando o ambiente MPI
17    MPI_Finalize();
18
19    return 0;
20 }
```

Listagem 2. Exemplo de código MPI.

2.3.1. Funcionalidades MPI

Os tipos de dados e funções disponíveis na biblioteca MPI permitem a comunicação entre processos de forma eficiente e segura, mas é necessário um conhecimento prévio sobre a biblioteca para utilizá-la corretamente. Com isso, será apresentado brevemente as principais funções e tipos de dados utilizados na biblioteca MPI.

As principais funções do MPI incluem:

- `MPI_Init`: Inicializa o ambiente MPI. Recebe argumentos da linha de comando e deve ser chamada antes de qualquer outra função MPI.
- `MPI_Finalize`: Finaliza o ambiente MPI. É a última função MPI a ser chamada, limpando recursos alocados.
- `MPI_Comm_size`: Retorna o número total de processos em um comunicador específico.

- `MPI_Comm_rank`: Retorna o identificador único (rank) do processo atual dentro do comunicador.
- `MPI_Send`: Envia uma mensagem para outro processo, especificando buffer, tipo de dado, destino e tag.
- `MPI_Recv`: Recebe uma mensagem de outro processo, armazenando os dados em um buffer.
- `MPI_Barrier`: Bloqueia a execução até que todos os processos no comunicador atinjam este ponto, garantindo sincronização.

Tabela 1. Principais Tipos de Dados MPI

Tipo de Dado	Descrição
<code>MPI_CHAR</code>	Representa um caractere
<code>MPI_INT</code>	Representa um inteiro
<code>MPI_FLOAT</code>	Representa um número de ponto flutuante de precisão simples
<code>MPI_DOUBLE</code>	Representa um número de ponto flutuante de precisão dupla
<code>MPI_LONG</code>	Representa um número inteiro longo
<code>MPI_UNSIGNED</code>	Representa um número inteiro sem sinal
<code>MPI_BYTE</code>	Representa um byte
<code>MPI_PACKED</code>	Representa dados empacotados

Além desses tipos de dados básicos, a biblioteca MPI permite a definição de tipos de dados derivados, que são combinações de tipos de dados básicos. Isso é útil para enviar e receber estruturas de dados complexas. As funções `MPI_Type_contiguous`, `MPI_Type_vector`, `MPI_Type_indexed`, entre outras, são utilizadas para criar esses tipos de dados derivados.

2.4. Stencil

O stencil é uma abordagem de programação paralela que consiste em aplicar uma operação a cada elemento de um conjunto de dados, utilizando os valores dos elementos vizinhos, conforme um raio de vizinhança predefinido. Essa técnica é particularmente útil em simulações numéricas que envolvem equações diferenciais parciais, como no caso da difusão de contaminantes em água. A computação baseada em stencil é altamente paralelizável, pois cada elemento pode ser processado independentemente, desde que os valores dos vizinhos estejam disponíveis. [Raúl de la Cruz and Araya-Polo 2015]

A abordagem do *stencil* é amplamente utilizado em diversas áreas da ciência computacional, como geofísica, astrofísica, física nuclear e oceanografia. Essas áreas demandam simulações computacionais em larga escala, com alta frequência de saída. Quando equações diferenciais parciais (EDPs) são empregadas, elas geralmente são resolvidas pelo método das diferenças finitas, utilizando o *stencil* para calcular os operadores diferenciais, que representam uma parcela significativa do tempo total de execução.

No uso do *stencil*, para cada ponto de um conjunto de dados, como em uma matriz bidimensional (2D), são considerados os pesos dos pontos vizinhos para resolver os operadores diferenciais espacialmente, como ilustrado na Figura 3. Dessa forma, replicando essas operações para todos os pontos do conjunto, é possível realizar a paralelização ideal, onde cada *thread* calcula o valor correspondente a um ponto. Isso possibilita

a paralelização eficiente do código, reduzindo significativamente o tempo de execução. [Raúl de la Cruz and Araya-Polo 2015]

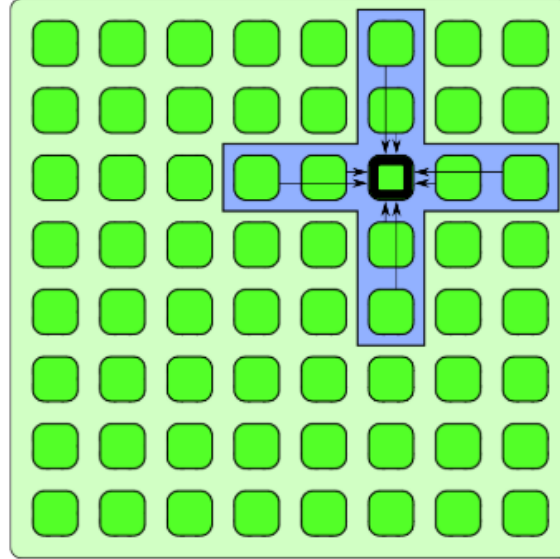


Figura 3. Exemplo de stencil 2D. [Stein and Griebler 2018]

3. Desenvolvimento

A simulação de um modelo de difusão de contaminantes em água pode ser equacionado por:

$$\frac{\partial C}{\partial t} + \mathbf{v} \cdot \nabla C = D \nabla^2 C$$

Onde:

- C é a concentração do contaminante.
- t é o tempo (em segundos).
- $\mathbf{v}$ é o vetor de velocidade do fluxo da água.
- D é o coeficiente de difusão do contaminante na água.
- $\nabla^2 C$ é a ataxa de variação da concentração no espaço.

Ou ainda, pode ser formulada de uma modo mais extenso

$$C_{i,j}^{(t+1)} = C_{i,j}^{(t)} + D \cdot \Delta t \cdot \frac{C_{i+1,j}^{(t)} + C_{i-1,j}^{(t)} + C_{i,j+1}^{(t)} + C_{i,j-1}^{(t)} - 4 \cdot C_{i,j}^{(t)}}{\Delta x^2}$$

A parte mais importante do código pode ser observada na Listagem 3, onde é resaltado os calculos para a concentração da iteração $i+1$.

```

1 void diff_eq(double **C, double **C_new) {
2     for (int t = 0; t < T; t++) {
3         for (int i = 1; i < N - 1; i++) {
4             for (int j = 1; j < N - 1; j++) {
5                 C_new[i][j] = C[i][j] + D * DELTA_T * (
6                     (C[i+1][j] + C[i-1][j] + C[i][j+1] + C[i][j-1] - 4 * C[i][j]) / (DELTA_X * DELTA_X)
7                 );
8             }
9         }
10    }

```

```

9     }
10    // Atualizar matriz para a proxima iteracao
11    double difmedio = 0.;
12    for (int i = 1; i < N - 1; i++) {
13        for (int j = 1; j < N - 1; j++) {
14            difmedio += fabs(C_new[i][j] - C[i][j]);
15            C[i][j] = C_new[i][j];
16        }
17    }
18    if ((t%100) == 0)
19        printf("iteracao %d - diferenca=%g\n", t, difmedio/((N-2)*(N-2)));
20 }
21 }

```

Listagem 3. Código da Equação de Difusão em C

Para alocar o máximo de espaço para a matriz bidimensional, foi necessário alocar memória de forma dinâmica para evitar que ocorra *segmentation fault (core dumped)* com valores altos, como mostra a Listagem 4. Isso também se aplica à matriz da próxima iteração, ou seja, *C_new*. Ambas as matrizes começam com 0 em todas as posições, e para a matriz *C*, há uma grande concentração no seu centro, igual a 1,0.

```

1 // Concentracao inicial
2 double **C = (double **)malloc(N * sizeof(double *));
3 if (C == NULL) {
4     fprintf(stderr, "Memory allocation failed\n");
5     return 1;
6 }
7 for (int i = 0; i < N; i++) {
8     C[i] = (double *)malloc(N * sizeof(double));
9     if (C[i] == NULL) {
10        fprintf(stderr, "Memory allocation failed\n");
11        return 1;
12    }
13 }

```

Listagem 4. Alocação de memória para a matriz

3.1. Paralelizando com o OpenMP

Para otimizar o código apresentado na Listagem 5, foi utilizado o conceito de *threads*, dividindo o cálculo da concentração de toda a matriz em *n threads*. A abordagem escolhida foi a utilização do *OpenMP*, conforme mostrado na Listagem 5. No primeiro conjunto de *for*, em *#pragma omp parallel for collapse(2) schedule(static)*, são usadas duas cláusulas: *collapse(2)*, que agrupa conjuntos de *loops* sequenciais (neste caso, dois), e *schedule(static)*, que divide as iterações do *for* de forma igual entre as *threads*.

Já no segundo conjunto de *for*, há mais uma cláusula: *reduction(+:difmedio)*, que faz a redução do *difmedio*. Assim, cada *thread* soma seu valor, e, no final, é necessário obter o valor resultante.

```

1 void diff_eq(double **C, double **C_new, int n_threads) {
2     omp_set_num_threads(n_threads);
3     for (int t = 0; t < T; t++) {
4         #pragma omp parallel for collapse(2) schedule(static)
5         for (int i = 1; i < N - 1; i++) {
6             for (int j = 1; j < N - 1; j++) {
7                 C_new[i][j] = C[i][j] + D * DELTA_T * (
8                     (C[i+1][j] + C[i-1][j] + C[i][j+1] + C[i][j-1] - 4 * C[i][j]) / (DELTA_X * DELTA_X)
9                 );
10            }
11        }
12
13        double difmedio = 0.;
14        #pragma omp parallel for collapse(2) reduction(+:difmedio) schedule(static)
15        for (int i = 1; i < N - 1; i++) {
16            for (int j = 1; j < N - 1; j++) {
17                difmedio += fabs(C_new[i][j] - C[i][j]);
18                C[i][j] = C_new[i][j];
19            }
20        }
21        if ((t%100) == 0)
22            printf("iteracao %d - diferenca=%g\n", t, difmedio/((N-2)*(N-2)));
23    }
24 }

```

Listagem 5. Código da Equação de Difusão em C com OpenMP

A configuração do número de *threads*, é passada por parâmetro no executável do programa.

Outra abordagem utilizada para otimizar o tempo de execução do processo foi a mudança na alocação de memória dinâmica, como mostrado na Listagem 6. Como pode ser visto na Figura 4, é realizada uma alocação para um *array* de *double* de N posições. Em seguida, é feita a alocação de outro *array* de N posições para cada posição do primeiro *array*. Para melhorar isso, foi alocado apenas um único *array* de $N*N$ posições, configurando a matriz desejada.

```
1 // Alocação dinâmica para as matrizes
2 double *C = (double *)malloc(N * N * sizeof(double));
3 double *C_new = (double *)malloc(N * N * sizeof(double));
```

Listagem 6. Alocação de memória para as matrizes de forma otimizada

Dessa forma, os cálculos para as concentrações também mudaram, mas apenas para atender às novas matrizes, como mostrado na Listagem 7.

```
1 void diff_eq(double *C, double *C_new, int n_threads) {
2     omp_set_num_threads(n_threads);
3     double delta_x_squared_inv = 1.0 / (DELTA_X * DELTA_X);
4
5     for (int t = 0; t < T; t++) {
6         #pragma omp parallel for schedule(static)
7         for (int i = 1; i < N - 1; i++) {
8             for (int j = 1; j < N - 1; j++) {
9                 C_new[i * N + j] = C[i * N + j] + D * DELTA_T * (
10                     (C[(i+1) * N + j] + C[(i-1) * N + j] + C[i * N + (j+1)] + C[i * N + (j-1)] - 4 * C
11                     [i * N + j]) *
12                     delta_x_squared_inv
13                 );
14             }
15         }
16
17         double difmedio = 0.0;
18         #pragma omp parallel for reduction(+:difmedio) schedule(static)
19         for (int i = 1; i < N - 1; i++) {
20             for (int j = 1; j < N - 1; j++) {
21                 difmedio += fabs(C_new[i * N + j] - C[i * N + j]);
22                 C[i * N + j] = C_new[i * N + j];
23             }
24         }
25
26         if ((t%100) == 0)
27             printf("iteracao %d - diferenca=%g\n", t, difmedio/((N-2)*(N-2)));
28     }
29 }
```

Listagem 7. Código otimizado da equação de difusão com OpenMP

3.2. Paralelizando com o CUDA

No vigente trabalho, foram realizados duas abordagens em CUDA. A primeira foi a paralelização do *stencil* 1D, e a segunda, do *stencil* 2D. Ambas as abordagens utilizam a mesma equação de difusão, mas com diferentes implementações para a matriz de concentração.

3.2.1. Stencil 1D

O stencil 1D utiliza um vetor unidimensional para realizar os cálculos, garantindo que não sejam acessados espaços de memória inválidos.

```

1 double diff_eq(double *C, double *C_new) {
2     double *d_C, *d_C_new;
3     double *swap, difmedio;
4
5     cudaMalloc( (void **) &d_C, SIZE);
6     cudaMalloc( (void **) &d_C_new, SIZE);
7
8     cudaMemcpy(d_C, C, SIZE, cudaMemcpyHostToDevice );
9     cudaMemcpy(d_C_new, C_new, SIZE, cudaMemcpyHostToDevice );
10
11     for (int t = 0; t < T; t++) {
12         diff_eq_kernel<<< (N * N + (THREADS_PER_BLOCK-1)) / THREADS_PER_BLOCK, THREADS_PER_BLOCK >>>(d_C, d_C_new, N);
13
14         cudaDeviceSynchronize();
15
16         swap = d_C;
17         d_C = d_C_new;
18         d_C_new = swap;
19
20         if (t%100==0) {
21             difmedio = .0;
22             cudaMemcpy(C, d_C, SIZE, cudaMemcpyDeviceToHost);
23             cudaMemcpy(C_new, d_C_new, SIZE, cudaMemcpyDeviceToHost);
24
25             for(int i=0; i<N*N; i++)
26                 difmedio += fabs(C[i] - C_new[i]);
27
28             printf("interacao %d - diferenca=%g\n", t, difmedio/((N-2)*(N-2)));
29         }
30     }
31
32     cudaMemcpy(C, d_C, SIZE, cudaMemcpyDeviceToHost);
33
34     cudaFree(d_C);
35     cudaFree(d_C_new);
36
37     return C[(N/2) * N + (N/2)];
38 }

```

Listagem 8. Chamada do *kernel*.

O trecho de código apresentado na Listagem 8 ilustra o preparo das variáveis para entrar no *kernel*, com as funções do *cuda*, como *cudaMalloc* e *cudaMemcpy*. As variáveis chaves são *d_C*, que contém a concentração atual, *d_C_new*, que calcula a próxima concentração da água, *swap*, uma variável auxiliar para troca entre o *d_C* e *d_C_new*. Ainda há a variável *difmedio*, para calcular a concentração média.

A função do *kernel*, *diff_eq_kernel*, é chamada com parâmetros que definem a quantidade de *threads* por bloco e de blocos, além das variáveis de concentração e a dimensão da matriz, que é considerada quadrada. Após a execução do *kernel*, as *threads* são sincronizadas e os ponteiros de memória são trocados. Quando a iteração é múltipla de 100, a diferença média é calculada e exibida. Por fim, as variáveis do *device* são liberadas, e a concentração no centro do meio é retornada.

```

1 __global__ void diff_eq_kernel(double *C, double *C_new, int n) {
2     int in dex = threadIdx.x + blockIdx.x * blockDim.x;
3
4     if (index >= n && index < n * (n - 1) && index % n != 0 && index % n != n - 1) {
5         C_new[index] = C[index] + D * DELTA_T * (
6             (C[index+1] + C[index-1] + C[index+N] + C[index-N] - 4*C[index]) / (DELTA_X * DELTA_X)
7         );
8     }
9 }

```

Listagem 9. Função *kernel*

A Listagem 9 apresenta a implementação do *kernel*, que realiza o cálculo da próxima concentração. A matriz é representada por um vetor unidimensional, e o índice da *thread* que executará o cálculo é determinado pela equação apresentada no código. Note que as bordas da matriz não são calculadas, uma vez que não possuem relação direta

com os valores adjacentes. A equação utilizada no *kernel* é equivalente àquela empregada no código sequencial.

3.2.2. Stencil 2D

O stencil 2D já utiliza uma matriz para realizar os cálculos, também com o cuidado de não acessar espaços de memória que não possam ser acessados. Contudo, a diferença entre o stencil 1D e o 2D é que, nessa implementação, no stencil 2D utilizou-se duas dimensões para encontrar o índice da *thread*. Especificamente as dimensões x e y da matriz do grid.

A implementação da chamada do *kernel* é bem similar à do stencil 1D, com pequenas variações entre as implementações, já que todos os laços percorrerão em outro laço, formando assim uma complexidade maior de cálculos. A implementação e chamada do *kernel* pode ser vista na Listagem 10.

```
1 int main() {
2     double *C, *C_new;
3     double *d_C, *d_C_new;
4     size_t size = MEM_SIZE_MATRIX;
5
6     C = (double*)malloc(size);
7     C_new = (double*)malloc(size);
8
9     if (C == NULL || C_new == NULL) {
10         fprintf(stderr, "Memory allocation failed\n");
11         return 1;
12     }
13
14     initialize_matrix(C, N);
15     initialize_matrix(C_new, N);
16
17     check_cuda_error(
18         cudaMalloc((void**)&d_C, size), "Failed to allocate device memory for C");
19     check_cuda_error(
20         cudaMalloc((void**)&d_C_new, size), "Failed to allocate device memory for C_new");
21
22     check_cuda_error(
23         cudaMemcpy(d_C, C, size, cudaMemcpyHostToDevice), "Failed to copy data from host to device");
24
25     dim3 block(BLOCK_SIZE_X, BLOCK_SIZE_Y);
26     dim3 grid((N + block.x - 1) / block.x, (N + block.y - 1) / block.y);
27
28     double difmedio;
29     double iStart = omp_get_wtime();
30
31     for (int t = 0; t < T; t++) {
32         diff_eq_kernel<<<grid, block>>>(d_C, d_C_new, N);
33         check_cuda_error(
34             cudaDeviceSynchronize(), "Kernel execution failed");
35
36         double *swapAux = d_C;
37         d_C = d_C_new;
38         d_C_new = swapAux;
39
40         if (t % 100 == 0) {
41             difmedio = 0.0;
42             check_cuda_error(
43                 cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost), "Failed to copy data from device to host");
44             check_cuda_error(
45                 cudaMemcpy(C_new, d_C_new, size, cudaMemcpyDeviceToHost), "Failed to copy data from device to host");
46
47             for (int i = 0; i < N * N; i++)
48                 difmedio += fabs(C[i] - C_new[i]);
49
50             printf("interacao %d - diferenca=%g\n", t, difmedio / ((N - 2) * (N - 2)));
51         }
52     }
53
54     double iElaps = omp_get_wtime() - iStart;
55
56     printf("\n<<<%d,%d>>>");
57     printf("%lf\n", iElaps);
58
59     check_cuda_error(
60         cudaMemcpy(C, d_C, size, cudaMemcpyDeviceToHost), "Failed to copy data from device to host");
61
62     cudaFree(d_C);
63     cudaFree(d_C_new);
64     free(C);
65     free(C_new);
```

```

66
67     return 0;
68 }

```

Listagem 10. Função principal com a chamada do *kernel* 2D.

Diante da Listagem 10, observa-se que, inicialmente, são alocados os ponteiros responsáveis por armazenar os valores das matrizes. Destaca-se a chamada da função `initialize_matrix`, que realiza a alocação de memória para as estruturas e inicializa o valor no centro da matriz `C`. Em seguida, a memória é alocada na GPU utilizando a função `cudaMalloc`, e os dados são transferidos do host para a GPU por meio da função `cudaMemcpy`.

Após essa etapa, configura-se a grade e os blocos utilizando a função `dim3`, uma estrutura que define as dimensões da grade e dos blocos. Por fim, um laço de repetição é executado para `T` iterações, em que o *kernel* `diff_eq_kernel` é invocado para calcular a difusão do contaminante na água. Em cada iteração, a concentração é calculada com base na técnica de programação Stencil. Ao término do laço, os dados são copiados da GPU para o host, e a memória alocada tanto na GPU quanto no host é liberada.

A Listagem 11 mostra a implementação do *kernel* que realiza os cálculos e também verifica os índices úteis para o stencil, já que deve-se descontar o raio do stencil para não acessar posições inválidas da matriz. Em contraste com o *kernel*, a Listagem 12 mostra as funções utilizadas para alocar e inicializar as matrizes e uma outra para verificar os erros de execução do CUDA.

```

1  __global__ void diff_eq_kernel(double *C, double *C_new, int n) {
2      int x = blockIdx.x * blockDim.x + threadIdx.x;
3      int y = blockIdx.y * blockDim.y + threadIdx.y;
4      int index = y * n + x;
5
6      if (x > 0 && x < n - 1 && y > 0 && y < n - 1) {
7          C_new[index] = C[index] + D * DELTA_T * (
8              (C[index + 1] + C[index - 1] + C[index + n] + C[index - n] - 4 * C[index]) / (DELTA_X * DELTA_X)
9          );
10     }
11 }

```

Listagem 11. *Kernel* 2D.

```

1  void initialize_matrix(double *m, int n) {
2      for (int i = 0; i < n; i++) {
3          for (int j = 0; j < n; j++) {
4              m[i * n + j] = 0.0;
5          }
6      }
7      m[(n / 2) * n + (n / 2)] = 1.0;
8  }
9
10 void check_cuda_error(cudaError_t err, const char *msg) {
11     if (err != cudaSuccess) {
12         fprintf(stderr, "CUDA Error: %s: %s\n", msg, cudaGetErrorString(err));
13         exit(EXIT_FAILURE);
14     }
15 }

```

Listagem 12. Inicialização e alocação das matrizes e verificação de erros CUDA.

Observe que, diferentemente do *Stencil* 1D desenvolvido no subcapítulo 3.2.1, o *kernel* criado para o *Stencil* 2D possui índices em função de duas dimensões, calculados a partir dos métodos `blockIdx.x`, `blockDim.x` e `threadIdx.x`, assim como dos métodos aplicados na dimensão `y`.

3.3. Paralelizando com o MPI

A implementação do MPI para o modelo de difusão foi realizada em duas abordagens: *Stencil 1D* (domínio unidimensional particionado) e *Stencil 2D* (domínio bidimensional com decomposição espacial). Ambas seguem o princípio de decomposição de domínio, onde cada processo MPI calcula uma sub-região da grade global e comunica valores de borda com processos vizinhos.

3.3.1. Stencil 1D

O presente trabalho implementa uma decomposição 1D do domínio, onde a grade global de tamanho $N \times N$ é dividida em faixas horizontais entre processos. Cada processo gerencia uma sub-região com $local_row = N/numprocs + 2$ linhas (incluindo áreas de *halo* para comunicação).

3.3.2. Stencil 2D

O código implementa uma decomposição 2D implícita, onde a grade é dividida em blocos retangulares. Cada processo calcula uma submatriz de $local_row$, com comunicação de halo bidirecional.

A comunicação de halo (Listagem 13) envolve o envio das linhas de borda superior e inferior para processos vizinhos usando comunicação não bloqueante, permitindo sobreposição entre computação e comunicação.

```
1 if (myid > 0) {  
2     MPI_Isend(C_local[1], N, MPI_DOUBLE, myid - 1, 0, comm, &requests[0]);  
3     MPI_Irecv(C_local[0], N, MPI_DOUBLE, myid - 1, 1, comm, &requests[1]);  
4 }  
5 if (myid < numprocs - 1) {  
6     MPI_Isend(C_local[local_rows - 2], N, MPI_DOUBLE, myid + 1, 1, comm, &requests[2]);  
7     MPI_Irecv(C_local[local_rows - 1], N, MPI_DOUBLE, myid + 1, 0, comm, &requests[3]);  
8 }
```

Listagem 13. Comunicação de halo no MPI 2D.

3.3.3. Estratégias de Otimização

Ambas as implementações utilizam técnicas avançadas do MPI para melhor desempenho:

- **Comunicação Não Bloqueante:** `MPI_Isend` e `MPI_Irecv` permitem sobreposição entre computação e comunicação.
- **Redução Global:** `MPI_Allreduce` agrega a diferença média de todos os processos de forma eficiente.
- **Alocação de Memória Otimizada:** No *Stencil 1D*, matrizes unidimensionais melhoram a localidade espacial.

4. Resultados

Os resultados foram divididos em tópicos para que pudesse ser feito uma análise mais detalhada de cada abordagem utilizada.

4.1. OpenMP

Para avaliar a implementação, utilizou-se as métricas clássicas de desempenho de aplicações paralelas, como tempo de execução, *speedup* e eficiência. Assim como requerido inicialmente pela atividade, realizou-se testes em uma grade de 2000x2000, variando o número de *threads* de 1 (execução sequencial) até 16. A média dos tempos foi calculada para cada caso, e os resultados comparativos entre as abordagens sequencial e paralela são apresentados nas Figuras 4 e 5.

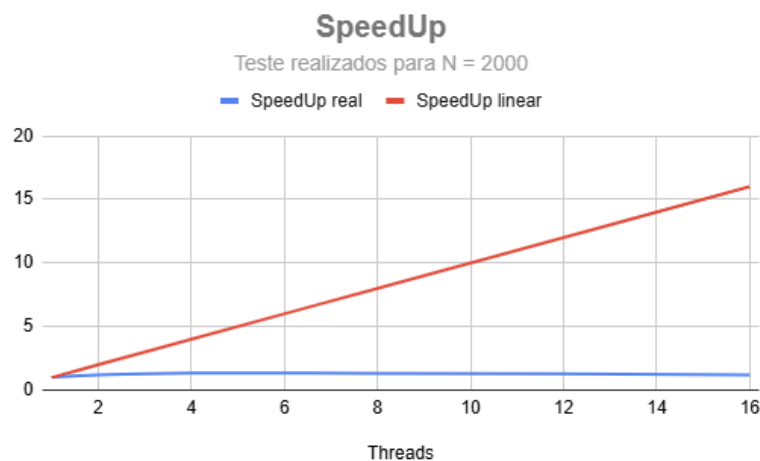


Figura 4. Gráfico de Speedup

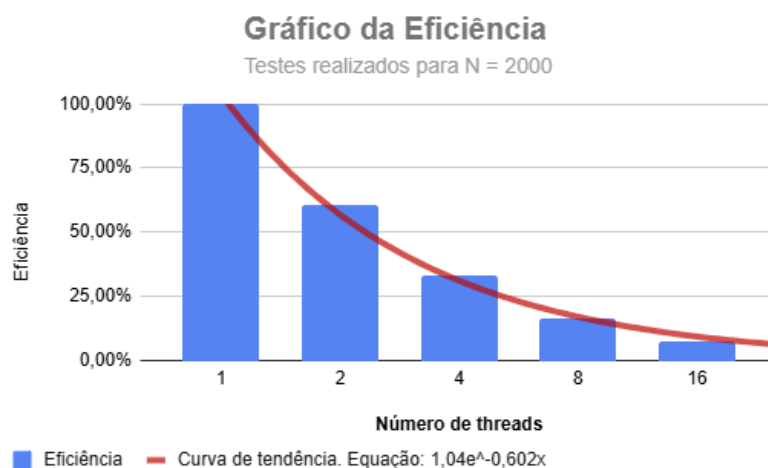


Figura 5. Gráfico de eficiência

Contudo, pode-se observar que os resultados não foram satisfatórios, já que o *speedup* não foi nem próximo do linear, e a eficiência foi caindo há medida do aumento do número de *threads*. Isso pode ser explicado pela natureza do problema, que possui um alto grau de paralelismo. Ainda assim, a implementação paralela obteve um desempenho superior ao sequencial, com um *speedup* máximo de 1.33 e eficiência de 33.25%.

4.2. CUDA

Após as implementações realizadas e mostradas na seção 3, foram realizados testes de desempenho para avaliar a eficácia da paralelização do modelo criado. As Figuras 6 e 7 mostram os resultados obtidos para o *Stencil* 2D e 1D, respectivamente.

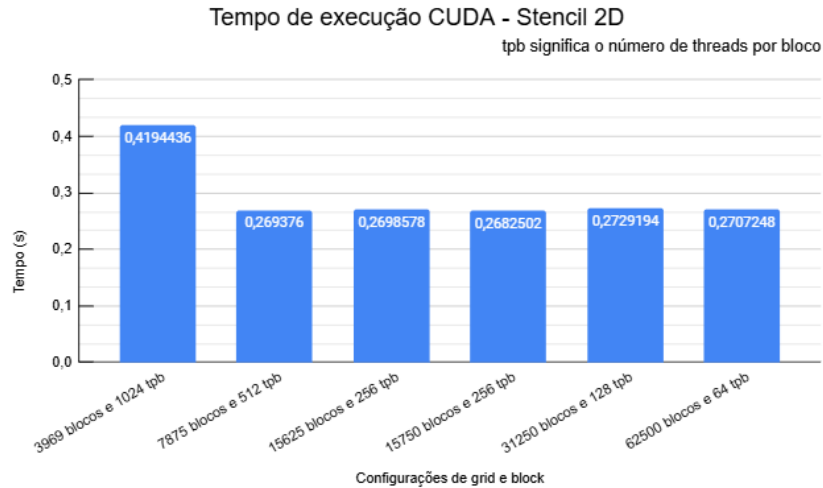


Figura 6. Gráfico de desempenho do *Stencil* 2D.

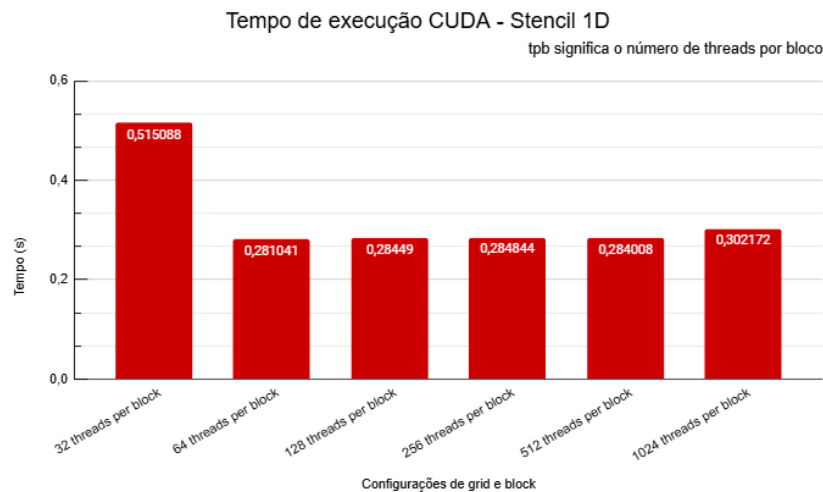


Figura 7. Gráfico de desempenho do *Stencil* 1D.

Diante os resultados da Figura 6 e 7, pode-se verificar que os resultados para diversas configurações de *grids* e *blocks* foram satisfatórios, já que para um problema com um número expressivo de iterações, o tempo de processamento total está na ordem de milissegundos. Contudo, é possível observar que o *Stencil* 2D obteve um desempenho superior ao *Stencil* 1D, o que pode ser justificado pela complexidade do cálculo, que envolve mais operações matriciais. Ainda sobre os resultados do *stencil* 2D, também pode-se observar que a configuração em que o número de blocos e o número de *threads* por bloco são de 7875 e 512, respectivamente.

Já sobre o *stencil* 1D, o desempenho também foi satisfatório mas inferior ao *stencil* 2D, o que pode ser explicado pela forma como os dados são acessados na memória. A configuração que apresentou melhor desempenho foi com 16384 blocos e 256 *threads* por bloco (64 *threads* por bloco), resultando em um tempo de execução médio de 281 milissegundos.

Comparando as duas implementações, pode-se observar que ambas as abordagens atingiram resultados expressivos em termos de desempenho. O *stencil* 2D demonstrou ser mais eficiente para este problema específico, possivelmente devido à melhor utilização da localidade espacial da memória e ao padrão de acesso aos dados mais otimizado. Por outro lado, o *stencil* 1D, embora com tempos de execução ligeiramente superiores, ainda manteve um excelente desempenho, demonstrando que ambas as estratégias são viáveis para a resolução do problema.

A Tabela 2 apresenta uma comparação entre as melhores médias das amostras quando implementadas em código sequencial, OpenMP e CUDA. Deve-se levar em consideração a confiabilidade dos resultados, já que o número de testes realizados foi superior a 30. Diante os resultados obtidos, a implementação que melhor performou foi a codificação utilizando a tecnologia CUDA, com um *speedup* de 80.1603 em relação à implementação sequencial. Por fim, a eficiência da implementação em CUDA não foi calculada, já que a modelagem CUDA apresenta características únicas, quando comparadas às demais. Portanto, o cálculo da eficiência para tal arquitetura deve ser diferente e não foi considerado para esse trabalho.

Tabela 2. Comparação entre as diferentes implementações

Implementação	Tempo (s)	Speedup	Eficiência
Sequencial	21.507	1.00	1.00
OpenMP	16.1707	1.3300	0.3325
CUDA 1D	0.2810	76.5374	-
CUDA 2D	0.2683	80.1603	-

Por fim, assim como realizada nos resultados para o *OpenMP*, a Figura 8 mostra o *speedup* obtido para a implementação com o *stencil* 1D, e a Figura 9 mostra o *speedup* obtido para a implementação com o *stencil* 2D. As ilustrações podem ser utilizadas para comparar os *speedups* encontrados em cada um dos paradigmas implementados de paralelismo. Para qualquer comparação, deve-se saber que o tempo de referência utilizado para calcular os *speedups* foi o tempo de execução da implementação sequencial, de 21.507 segundos.

4.3. MPI

Os resultados obtidos no experimento demonstram diferenças pouco significativas entre as implementações 1D e 2D no que se refere ao desempenho da paralelização. A decomposição 1D apresentou uma execução mais eficiente, com menor sobrecarga de comunicação, enquanto a 2D obteve uma redução de tempo mais expressiva, mas enfrentou desafios relacionados à escalabilidade devido à troca de dados entre múltiplos processos.

Ao analisar o *speedup*, a implementação 1D atingiu um valor próximo ao ideal

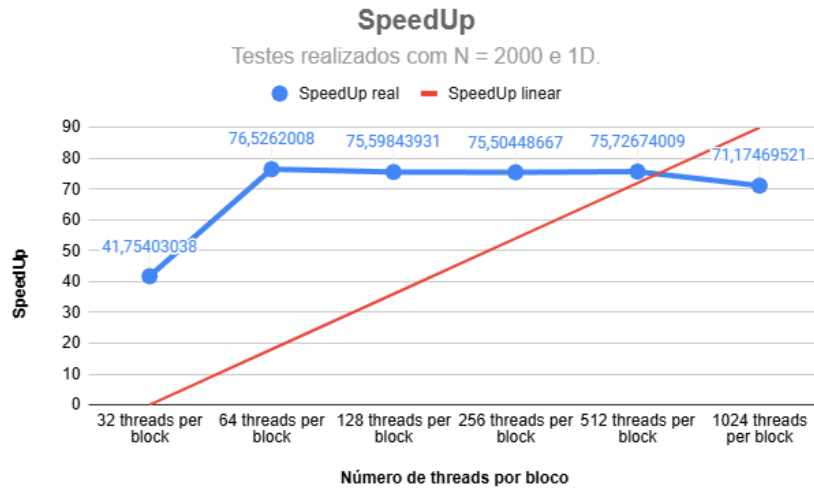


Figura 8. Gráfico de *speedup* para o *stencil* 1D.

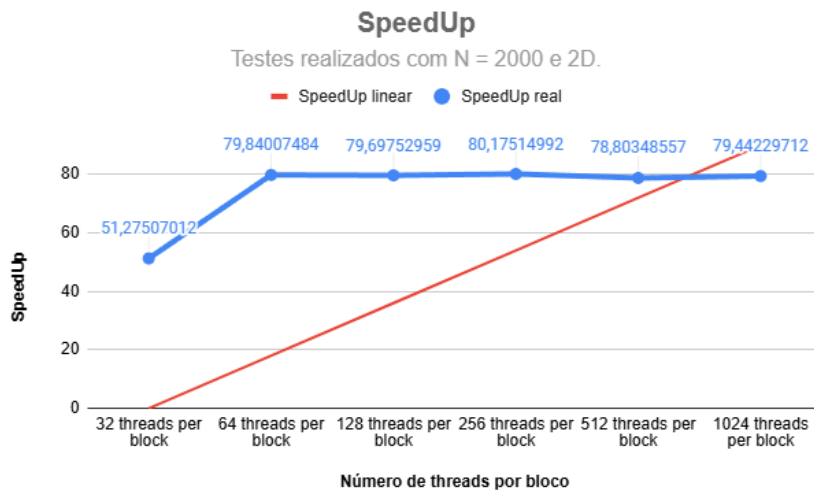


Figura 9. Gráfico de *speedup* para o *stencil* 2D.

com quatro processos, o que indica uma boa escalabilidade até esse número de processos. Já a decomposição 2D obteve um *speedup* real máximo de aproximadamente 4,57 vezes com 4 processos. Isso pode ser explicado pelo aumento no número de vizinhos na memória, já que a abordagem 2D possui vizinhos em quatro direções.

Já sobre os tempos, as Figuras 12 e 13 mostram os tempos de execução para as implementações do *stencil* 1D e 2D, respectivamente. Com base nos resultados, pode-se observar que os tempos para 1D e 2D ficaram bem próximos, com uma pequena vantagem para o *stencil* 1D. Contudo, a diferença entre os tempos de execução não foi significativa, o que sugere que ambas as implementações são eficientes e apresentam um bom desempenho.

Conclui-se, portanto, que a decomposição 1D foi a abordagem sutilmente mais eficiente para o problema analisado, permitindo um ganho próximo ao ideal com o mínimo de sobrecarga, para o número de processos menor ou igual a 4. A decomposição 2D,

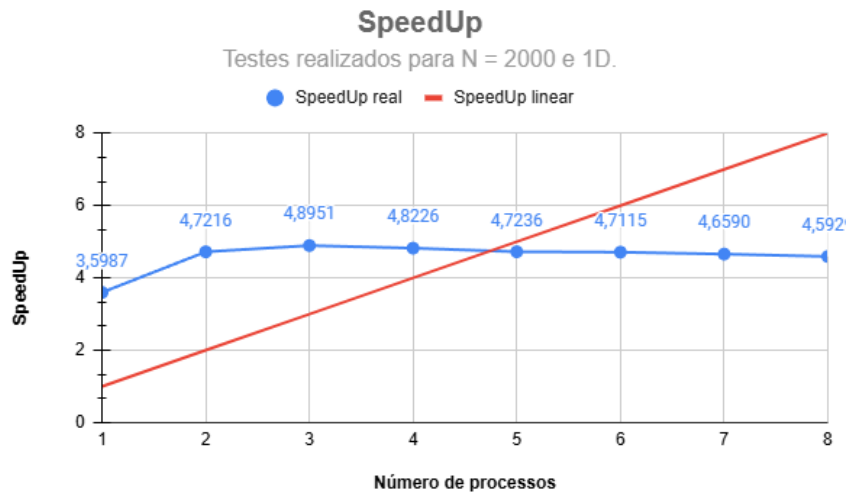


Figura 10. Gráfico de *speedup* para o *stencil* 1D.

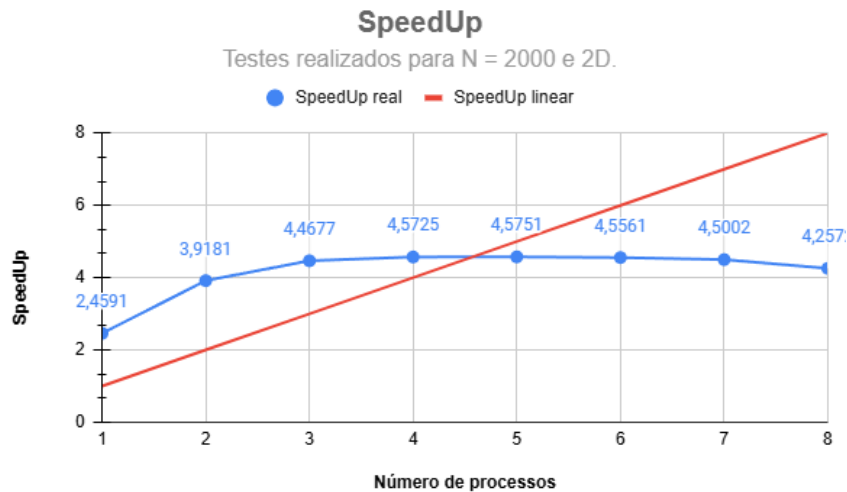


Figura 11. Gráfico de *speedup* para o *stencil* 2D.

embora tenha apresentado uma redução de tempo mais significativa, teve sua escalabilidade limitada pela necessidade de sincronização entre os processos. A escolha entre as abordagens deve levar em conta a relação entre simplicidade e potencial de paralelismo, com otimizações na comunicação podendo melhorar a performance da decomposição 2D em cenários mais amplos. Contudo, deve-se considerar que os valores de tempo basais utilizados para calcular o *speedup* foi o tempo de execução da implementação sequencial, que foi de 21.507 segundos. Pensou-se nessa abordagem para que houvesse uma justiça nas comparações.

4.4. Comparação entre as Implementações

Devido as diferentes abordagens de paralelização, a comparação de *benchmark* das metodologias é de difícil análise. Contudo, a Tabela 2 apresenta uma comparação entre as melhores médias das amostras quando implementadas em código sequencial, OpenMP, CUDA e MPI.

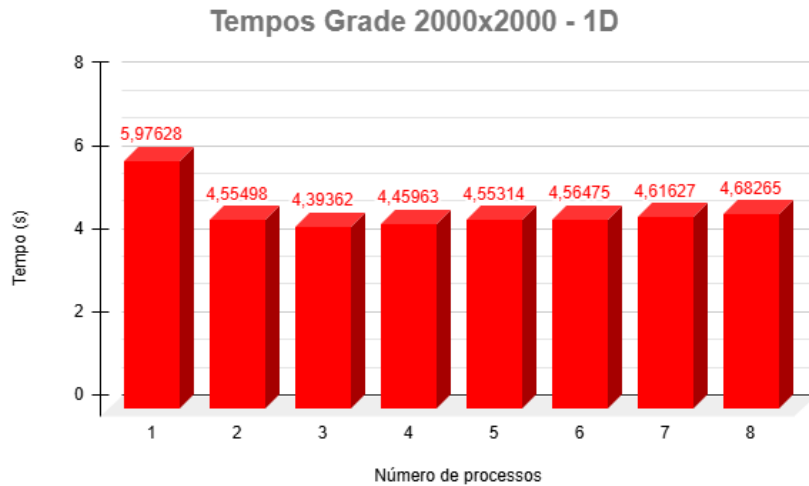


Figura 12. Gráfico do tempo para o *stencil* 1D.

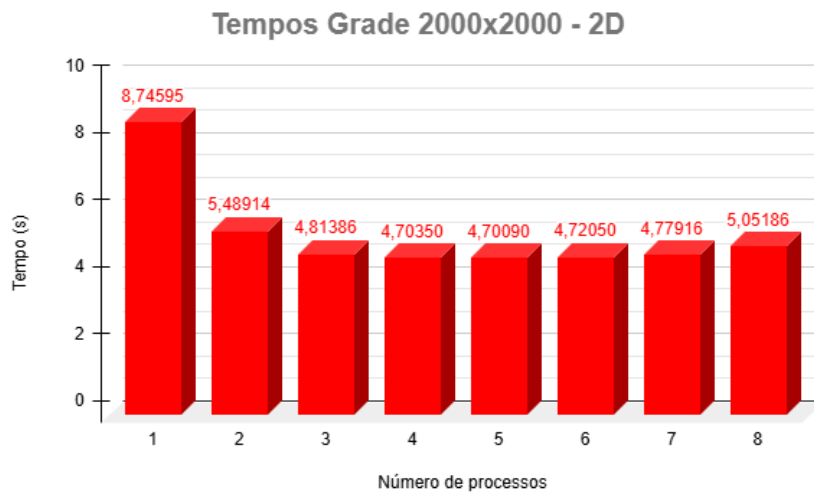


Figura 13. Gráfico do tempo para o *stencil* 2D.

5. Conclusão

Devido aos resultados sumarizados na Tabela 3, pode-se concluir que a implementação em CUDA 2D foi a mais eficiente, com um *speedup* de 80.1603 em relação à implementação sequencial. A implementação em OpenMP obteve resultados pouco satisfatórios, obtendo, no melhor caso, uma eficiência inferior à metade da referência utilizada. Já as implementações em MPI, embora tenham apresentado um desempenho satisfatório, não foram tão eficientes quanto as implementações em CUDA. A implementação 1D obteve um *speedup* de 4.8951, enquanto a implementação 2D obteve um *speedup* de 4.5751. Contudo, deve-se levar em consideração que a máquina utilizada para realizar os testes não tinha uma GPU capaz de realizar os cálculos em CUDA, o que pode ter influenciado nos resultados obtidos, pois usamos a *engine* T3 do *Google Colab*.

No mais, a implementação em CUDA foi a mais eficiente para o problema proposto, possuindo resultados expressivos até com o aumento no número de iterações e um

Tabela 3. Comparação entre as diferentes implementações

Implementação	Tempo (s)	Speedup	Eficiência
Sequencial	21.507	1.00	1.00
OpenMP	16.1707	1.3300	0.3325
CUDA 1D	0.2810	76.5374	-
CUDA 2D	0.2683	80.1603	-
MPI 1D	4.3936	4.8951	1.6316
MPI 2D	4.7009	4.5751	0.9150

aumento no número de iterações também. Já a abordagem em MPI, os resultados obtidos foram ótimos, pois foram executados em um *laptop* de uso pessoal e obteve um *speedup* de quase cinco vezes em relação à implementação sequencial.

6. Referências

References

- Chapman, B., Jost, G., and van der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming*. The MIT Press.
- Cheng, J., Grossman, M., and McKercher, T. (2014). *Professional CUDA C Programming*. John Wiley and Sons, Inc.
- Gropp, W. and Lusk, E. (2017). *Scientific Programming and Computer Architecture*. The MIT Press.
- Gropp, W., Lusk, E., and Skjellum, A. (2014). *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 3rd edition.
- McKennon, J. (2013). Cuda parallel thread management.
- MPI Forum (2021). *MPI: A Message-Passing Interface Standard Version 4.0*. Acesso em: 14 jan. 2023.
- NVIDIA Corporation (2012). *CUDA C PROGRAMMING GUIDE*.
- NVIDIA Corporation (2018). *NVIDIA TURING GPU ARCHITECTURE*. NVIDIA.
- Owens, J. D., Houston, M., Luebke, D., Green, S., Stone, J. E., and Phillips, J. C. (2008). Gpu computing. *Proceedings of the IEEE*, 96(5):879–899. Retrieved from <http://dx.doi.org/10.1109/JPROC.2008.917757>.
- Raúl de la Cruz and Araya-Polo, M. (2015). Modeling stencil computations on modern hpc architectures. *Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS14)*.
- Stein, C. and Griebler, D. (2018). *Programação Paralela para GPU em Aplicações de Processamento de Stream*. PhD thesis.